PPF WEEK 9 — DSA MINI PROJECT REPORT

STUDENT RECORD SEARCH SYSTEM USING BINARY SEARCH TREE (BST)

---

1. ABSTRACT

The Student Record Search System is a menu-driven application developed using the C programming language and the Binary Search Tree (BST) data structure. The main purpose of this project is to store, search, insert, delete, and display student records efficiently.

Each student record contains a unique enrolment ID, student name, and department. The enrolment ID is used as the key for organizing records in the Binary Search Tree.

The system supports insertion of new student records, searching for students by enrolment ID, deletion of existing records, and displaying records through inorder, preorder, and postorder traversals. It also prevents duplicate enrolment IDs and provides a comparison between balanced-looking and sorted insertion orders.

This project demonstrates important data structure concepts, including binary trees, recursive algorithms, tree traversal, node deletion, tree height, and time complexity. It also explains how the shape of a BST affects its efficiency and how a self-balancing tree such as an AVL tree can improve performance.

2. INTRODUCTION

Managing student records is an important task in educational institutions. A student record system needs to store information, retrieve records quickly, and allow modifications when required.

If student records are stored in an unsorted array, searching may require checking every record. A Binary Search Tree provides another way to organize records according to their keys.

A Binary Search Tree is a non-linear data structure in which each node contains a key and pointers to its left and right children.

For this project, the enrolment ID is used as the key. Records with smaller IDs are stored in the left subtree, while records with larger IDs are stored in the right subtree.

The project is implemented in C using structures, pointers, dynamic memory allocation, and recursive functions.

3. PROBLEM STATEMENT

To develop a Student Record Search System using a Binary Search Tree in the C programming language.

The system must store student enrolment IDs, names, and departments. It should support insertion, searching, deletion, and sorted display of records.

The system must reject duplicate enrolment IDs, demonstrate all three tree traversals, handle different deletion cases, and compare the performance of balanced-looking and sorted insertion orders.

4. OBJECTIVES

The main objectives of the project are:

1. To understand the structure and properties of a Binary Search Tree.
2. To store student records using dynamically allocated tree nodes.
3. To implement insertion and reject duplicate enrolment IDs.
4. To search for student records using their enrolment IDs.
5. To delete nodes with zero, one, or two children.
6. To implement inorder, preorder, and postorder traversals.
7. To display student records in ascending enrolment-ID order.
8. To compare the number of key comparisons for different insertion orders.
9. To understand the average-case and worst-case time complexity of BST operations.
10. To understand the importance of balancing a tree.

5. SCOPE OF THE PROJECT

The project provides a basic student record management system for demonstrating data structure operations.

Its scope includes:

- Adding student records.
- Searching for a student by enrolment ID.
- Deleting an existing student record.
- Displaying records in sorted order.
- Performing different tree traversals.
- Loading ten predefined sample student records.
- Comparing two different BST insertion orders.

The project is designed for educational purposes. It demonstrates how a BST can be used to organize and retrieve data.

The current implementation stores records in memory. It does not permanently save records to a file or database.

6. TECHNOLOGIES USED

Component	Description
Programming Language	C
Data Structure	Binary Search Tree
Compiler	GCC
Code Editor	Visual Studio Code
Memory Management	Dynamic memory allocation using malloc() and free()
Data Organization	Structures and pointers
Algorithms	Insertion, searching, deletion, and tree traversal

7. DATA STRUCTURE USED

7.1 Binary Search Tree

A Binary Search Tree is a binary tree that follows an ordering property.

For every node:

- All keys in its left subtree are smaller than the node's key.
- All keys in its right subtree are larger than the node's key.
- Duplicate keys are not permitted in this project.

These rules allow the program to determine which subtree to search when looking for a particular enrolment ID.

7.2 Student Node Structure

Each node stores the following information:

- Enrolment ID
- Student name
- Department
- Pointer to the left child
- Pointer to the right child

The following structure is used in the program:

```
struct Student
{
int enrolmentID;
char name[50];
char department[50];
struct Student *left;
struct Student *right;
};
```

The `enrolmentID` field stores the unique key. The `name` and `department` fields store the student's information.

The `left` pointer connects to the left child, and the `right` pointer connects to the right child.

A pointer is set to NULL when the node has no corresponding child.

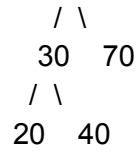
7.3 Example of a BST

Consider the following enrolment IDs:

50, 30, 70, 20, 40

The BST is:

50



Here:

- 50 is the root node.
- 30 and 70 are children of 50.
- 20 and 40 are children of 30.
- All left-side keys are smaller than their parent keys.
- All right-side keys are larger than their parent keys.

This structure follows the BST ordering property.

8. SYSTEM DESIGN

8.1 System Workflow

The program follows these steps:

1. Start the application.
2. Display the menu.
3. Accept the user's choice.
4. Execute the selected BST operation.
5. Display the result.
6. Return to the menu.
7. Continue until the user selects Exit.

8.2 Menu Options

The program provides the following options:

Option	Operation
1	Insert Student
2	Search Student
3	Delete Student

4	Inorder Traversal
5	Preorder Traversal
6	Postorder Traversal
7	Insert 10 Sample Records
8	Compare Insertion Orders
9	Exit

8.3 Functional Requirements

The system must:

- Accept student information.
- Reject duplicate enrolment IDs.
- Locate existing student records.
- Report when a student record does not exist.
- Delete records while maintaining the BST property.
- Display records using three traversal methods.
- Compare insertion orders using key comparisons.

9. ALGORITHMS

9.1 Insertion Algorithm

Purpose: To insert a new student record into the BST.

Steps:

1. Check whether the current node is NULL.
2. If it is NULL, create a new node.
3. If the new enrolment ID is smaller than the current ID, move to the left subtree.
4. If the new enrolment ID is larger than the current ID, move to the right subtree.
5. If the ID already exists, reject the insertion.
6. Return the updated root pointer.

The insertion operation maintains the BST ordering property.

9.2 Searching Algorithm

Purpose: To find a student record using an enrolment ID.

Steps:

1. Start from the root.
2. Compare the required ID with the current node's ID.
3. If both IDs are equal, return the matching student record.
4. If the required ID is smaller, move to the left child.
5. If the required ID is larger, move to the right child.
6. If the current node becomes NULL, report that the record was not found.

Searching avoids visiting subtrees that cannot contain the required key.

9.3 Deletion Algorithm

Purpose: To remove a student record while maintaining the BST property.

Deletion has three main cases.

Case 1: Deleting a Leaf Node

A leaf node has no children.

Example:

```
  30
 /  \
20    40
```

If node 20 is deleted, it can simply be removed.

The parent node's left pointer becomes NULL.

Case 2: Deleting a Node with One Child

A node with one child has either a left child or a right child.

Example:

```
 30
   \
    40
```

\
50

If node 40 is deleted, node 50 takes its place.

The remaining subtree stays connected to the parent.

Case 3: Deleting a Node with Two Children

A node with two children cannot simply be removed without reorganizing the tree.

The program finds the smallest node in the right subtree. This node is called the inorder successor.

The program copies the successor's enrolment ID, name, and department into the node being deleted. It then removes the successor from its original position.

This preserves the BST ordering property.

9.4 Inorder Traversal

Order: Left → Root → Right

Steps:

1. Traverse the left subtree.
2. Display the current node.
3. Traverse the right subtree.

Inorder traversal displays enrolment IDs in ascending order.

9.5 Preorder Traversal

Order: Root → Left → Right

Steps:

1. Display the current node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Preorder traversal displays the root before its children.

9.6 Postorder Traversal

Order: Left → Right → Root

Steps:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Display the current node.

Postorder traversal displays the root after its children.

9.7 Comparison of Insertion Orders

The project compares two sequences of enrolment IDs.

Balanced-looking order:

40, 20, 60, 10, 30, 50, 70

Sorted order:

10, 20, 30, 40, 50, 60, 70

The first sequence produces a relatively balanced tree. The second produces a right-skewed tree.

The program counts key comparisons performed during insertion to demonstrate the effect of tree shape on performance.

10. IMPLEMENTATION DETAILS

10.1 Structure Definition

The `Student` structure represents each node in the BST.

It stores the student's enrolment ID, name, department, and pointers to the left and right children.

10.2 Dynamic Memory Allocation

The program uses `malloc()` to allocate memory for a new node.

The `free()` function releases memory when nodes are deleted or when the program exits.

This allows the BST to grow and shrink as records are inserted and deleted.

10.3 Recursive Functions

Insertion, inorder traversal, preorder traversal, postorder traversal, deletion, and memory cleanup use recursion.

A recursive function calls itself to process a smaller part of the tree.

For example, inorder traversal recursively processes the left subtree, displays the current node, and then processes the right subtree.

10.4 Duplicate ID Handling

Every enrolment ID must be unique.

Before inserting a new record, the program searches for the ID. If the ID already exists, it rejects the new record.

The sample-record function also checks whether each sample ID is already present. This prevents duplicate records when option 7 is selected repeatedly.

10.5 Sample Records

The program provides ten sample student records.

Enrolment ID	Name	Department
50	Aman	CSE
30	Priya	ECE
70	Rahul	IT
20	Simran	CSE
40	Karan	IT
60	Neha	ECE
80	Rohit	CSE
10	Anjali	IT

35	Vikas	ECE
65	Pooja	CSE

These records can be inserted using menu option 7.

11. TEST CASES

The following test cases can be used to verify the program.

Test Case 1: Insert Sample Records

Input: Select option 7.

Expected result: Ten sample records are inserted when the tree is initially empty.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 2: Successful Search

Input: Select option 2 and enter enrolment ID 50.

Expected result:

Student Found!

Enrolment ID: 50
Name: Aman
Department: CSE

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 3: Unsuccessful Search

Input: Search for enrolment ID 999.

Expected result:

Student record not found.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 4: Duplicate Enrolment ID

Input: Try inserting enrolment ID 50 when it already exists.

Expected result:

Duplicate enrolment ID! Record not inserted.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 5: Delete a Leaf Node

Input: Delete enrolment ID 10 from the sample tree.

Expected result: ID 10 is deleted, and all other records remain available.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 6: Delete a Node with One Child

Input: Delete enrolment ID 60 from the original sample tree.

Expected result: ID 60 is deleted, and its child, ID 65, remains in the tree.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 7: Delete a Node with Two Children

Input: Delete enrolment ID 70 from the original sample tree.

Expected result: ID 70 is deleted, and the BST ordering property is maintained.

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 8: Inorder Traversal

Input: Select option 4.

Expected result: Student records are displayed in ascending order of enrolment ID.

For the original sample tree, the ID sequence is:

10, 20, 30, 35, 40, 50, 60, 65, 70, 80

Actual result: To be filled after testing.

Status: To be filled after testing.

Test Case 9: Insertion Order Comparison

Input: Select option 8.

Expected result: The program displays the number of key comparisons for balanced-looking and sorted insertion orders.

Actual result: To be filled after testing.

Status: To be filled after testing.

12. TIME COMPLEXITY ANALYSIS

Let n represent the number of nodes in the BST and h represent its height.

12.1 Insertion

Average case: O(log n)

Worst case: O(n)

Insertion is efficient when the tree is reasonably balanced. In a skewed tree, a new node may need to be placed after traversing many existing nodes.

12.2 Searching

Average case: O(log n)

Worst case: O(n)

Searching follows one path from the root toward a matching node or an empty child position.

12.3 Deletion

Average case: O(log n)

Worst case: O(n)

Deletion first searches for the node and then handles its children. The total operation depends on the tree height.

12.4 Tree Traversals

Inorder: O(n)

Preorder: O(n)

Postorder: O(n)

Each traversal visits every node once.

12.5 Space Complexity

The BST requires O(n) space to store n student records.

Recursive operations require additional stack space proportional to the tree height, O(h).

In a balanced tree, the height is approximately O(log n). In a skewed tree, it can reach O(n).

13. BALANCED BST VS. SKEWED BST

A balanced-looking BST has a relatively small height, allowing operations to reach the required node with fewer comparisons.

A skewed BST resembles a linked list. When IDs are inserted in sorted order, each new node may become the right child of the previous node.

This increases the number of comparisons required for searching, insertion, and deletion.

A regular BST does not automatically maintain its balance.

An AVL tree is a self-balancing BST. It uses rotations after insertions and deletions to maintain a balanced structure.

This keeps search, insertion, and deletion at O(log n) time.

14. ADVANTAGES

1. Student records are organized according to enrolment ID.
2. Searching can be efficient when the tree is balanced.
3. Inorder traversal displays records in sorted order.
4. Duplicate enrolment IDs are rejected.
5. Dynamic memory allocation allows records to be added as required.
6. The program demonstrates all three common tree traversals.
7. Deletion handles leaf nodes, nodes with one child, and nodes with two children.
8. The comparison function demonstrates how insertion order affects BST performance.

15. LIMITATIONS

1. Records are stored in memory and are not permanently saved.
2. The BST does not automatically rebalance itself.
3. Sorted insertion can produce a skewed tree.
4. The system supports basic student information only.
5. The program does not provide a graphical user interface.
6. Search and modification performance can decrease when the tree becomes highly skewed.

16. FUTURE SCOPE

The project can be extended in several ways:

1. Add file handling to save student records permanently.
2. Load saved records when the program starts.
3. Implement an AVL tree to maintain balance automatically.
4. Add options to update student names and departments.
5. Add validation for enrolment IDs and other inputs.
6. Display the height of the tree.

7. Compare BST and AVL tree performance using larger datasets.
8. Add a graphical interface for easier interaction.

17. CONCLUSION

The Student Record Search System demonstrates the use of a Binary Search Tree to organize and manage student records in the C programming language.

The project implements insertion, searching, deletion, and inorder, preorder, and postorder traversals. It also handles duplicate enrolment IDs and demonstrates the three main cases of node deletion.

The insertion-order comparison illustrates that the structure of a BST affects its efficiency. A reasonably balanced tree can support operations in O(log n) average time, whereas a skewed tree can require O(n) time in the worst case.

Overall, this project provides practical experience with binary trees, pointers, dynamic memory allocation, recursion, and algorithmic complexity.